

Advanced Natural Language Processing

Assignment 4: Transformer Architecture

Credits:

1 Introduction

In this assignment, you will work with an already existing repository that includes all the elements of a GPT model. As data to work with, we will use the short novel **Frankenstein** by **Mary Shelley**, written in 1818. (Consider how using this specific dataset might affect your results.)

You should implement all parts of this exercise using only `python`, the `pytorch` library, and its standard dependencies (e.g., `numpy`).

2 Getting started

You should start by unpacking the archive `assignment4.zip`. This results in a directory with the following structure:

```
- release_version/
--- data/
----- __init__.py
----- frankenstein.txt
----- frankenstein_short.txt
----- die_automata.txt
--- model/
----- __init__.py
----- model_gpt.py
----- utils.py
--- requirements.txt
--- assignment4.py
--- pretrained.py
--- config.json
--- train.py
```

3 Starter Code

To start, you should first install the libraries specified in `requirements.txt` by running the command `pip install -r requirements.txt` in your terminal. (You will likely want to do this in a separate python environment created specifically for this assignment, either via `venv` or `conda`, or directly in your IDE. Familiarise yourself with these packages, as this will come in handy for later projects.) **Beware that some parts of this assignment might require hours of compute time, so plan accordingly.**

In `assignment4.py`, you will find the main function of our code, in charge of loading the data, splitting it into training and evaluation, initializing our model, and loading dataloaders.

The code for this assignment is based on the repository accompanying the book “Build a Large Language Model (From Scratch)” by Sebastian Raschka (published by Manning, 2024).

The datasets for this assignment were taken from the Project Gutenberg website. You can access the complete Frankenstein by Mery Shelly’s (1818) book [here](#).

Die Automata (1819) is a short story by E. T. A. Hoffmann, and can be found [here](#).

3.1 Architecture [20 pts]

Take a look at `model_gpt.py` and try to understand its components, which implement the core transformer architecture. You are expected to create a diagram in which you explain the architecture of the model provided to you, labeling key components like the **multi-head self-attention block** and the **feed-forward network**. Additionally, show in the diagram the values of any hyperparameters defined in the model that can be highlighted.

To see examples of how to explain and diagram the architecture, check out this [blog post](#) by Sebastian Raschka.

3.2 Code interpretation [10 pts]

In the file `model_gpt.py`, provide detailed comments for each line of code, clarifying each function, and describing how the components interact within the overall model architecture.

3.3 Evaluate the model [10 pts]

Analyze and evaluate the learning process of the given model, on the basis of a plot of the loss function across the epochs for both the training and evaluation sets. What can you conclude about the process, based on the graph? What might be the cause of the behavior you are seeing?

Implement the plotting functionality in the provided `plot_losses` function in `train.py` file. You should also explain the evolution of the printed generated text during training.

As a second part of this task, implement functionality for calculating the perplexity of your trained model on a test corpus. We provide two new test corpora, `frankenstein_short.txt`, and `die_automata.txt`. Do so by implementing the code in `assignment4.py` under the `--perplexity` flag.

Remember that perplexity measures how well a language model predicts a sequence of tokens (how “perplexed” it is by it) and can be calculated as:

$$\text{Perplexity} = \exp \left(- \frac{1}{N} \sum_{i=1}^N \log p_{\theta}(x_i \mid x_1, x_2, \dots, x_{i-1}) \right)$$

where each component has the following meaning:

Component	Meaning
x_i	The i -th token in the test sequence.
N	Total number of tokens in the test text.
$p_\theta(x_i \mid x_1, \dots, x_{i-1})$	Probability assigned by the model with parameters θ to token x_i given all previous tokens.
$\log p_\theta(\cdot)$	Natural logarithm of the probability, converting product of probabilities into a sum (log-likelihood).
$-\frac{1}{N} \sum_{i=1}^N \log p_\theta(x_i \mid \dots)$	Average negative log-likelihood (NLL) over all tokens; measures how surprised the model is by the text.
$\exp(\cdot)$	Exponentiating the average NLL to obtain perplexity; intuitively the effective number of choices the model makes per token.

What can you observe from your results? Include a small section about them in your report, including a table, and explain if your results could be improved, and if so, how.

3.4 Decoding [20 pts]

With the model architecture established, we now turn to its use during text generation. Your task is to extend `generate_text_simple()` in `model/utils.py`. At the moment, this function implements greedy decoding, selecting at each timestep the token with the highest predicted probability. We expect you to implement two additional decoding methods:

- **Top- k decoding:** only the k most probable candidate tokens are retained, their probabilities are renormalized, and the next token is sampled from this restricted distribution.
- **One method of your choice** (e.g., top- p sampling, beam search, ...).

You are not allowed to use a predefined algorithm from pytorch. But you are encouraged to check how other people have implemented these techniques. However, it is mandatory to reference the resources where you got the implementation idea.

3.5 Hyperparameters [25 pts]

Experiment with different hyperparameter configurations by modifying the `config.json` file.

Please include in your report a discussion of your findings (for example, how loss and the generated text are affected), as well as a summary table listing all models you trained with their corresponding hyperparameters. Your discussion should address both the interpretation of the results and the plots you generate.

The `GPT_CONFIG` list may include architectural examples such as:

```
{ "gpt2-small (124M)": { "emb_dim": 768, "n_layers": 12, "n_heads": 12 },
  "gpt2-medium (355M)": { "emb_dim": 1024, "n_layers": 24, "n_heads": 16 },
  "gpt2-large (774M)": { "emb_dim": 1280, "n_layers": 36, "n_heads": 20 },
  "gpt2-xl (1558M)": { "emb_dim": 1600, "n_layers": 48, "n_heads": 25 }, }
```

In your discussion, describe how these parameters influence the model's performance, both in terms of token generation quality and loss behavior.

Note: Larger hyperparameters imply longer training times. Training the largest configurations above may require several hours, and some machines may not be able to run them

at all. You are encouraged to choose model sizes appropriate to your computational resources.

3.6 Using a pretrained GPT-2 model [15 points]

Training large language models from scratch requires substantial compute and time, so it is common to rely on models that have already been trained on large-scale datasets. In this task, you will download and use the pretrained `gpt2-small` model hosted on HuggingFace, with the pretrained weights available at:

<https://huggingface.co/rasbt/gpt2-from-scratch-pytorch/resolve/main/gpt2-small-124M.pth>

This task is to be implemented in `pretrained.py` by completing the `load_pretrained_gpt()` function. The goal is to download the checkpoint if it is not already present locally, create a model with the correct GPT-2 configuration, load the weights, move the model to the appropriate device, and generate a short text sample from a given `start_context`. Short guidance is provided in the comments of the function.

In the report, include a short discussion comparing the output of the pretrained model with the one you trained from scratch.

4 Submission

For the submission of this assignment:

- Upload your `assignment4_LASTNAME.zip` file on Moodle.
- Upload the same file to a GitUP repository. Add the tutors as *reporters* (users `@castilloma` and `@danzarovir`), and include the last commit hash of your project in the discussion report. (Please note these steps are no longer optional. You MUST upload your project to GitUP)
- Write the discussion report in $\text{\LaTeX}$. It must be no longer than 2 pages, not counting plots and diagrams.